

Ouvidoria Inteligente: triagem semântica de manifestações

Felipe Antônio Luna Medeiros (trabalho individual)

Tendências em Ciência da Computação · UNIPÊ · Prof. Ricardo Roberto de Lima

1. Objetivo e entregáveis

A Ouvidoria de um município faz a triagem das manifestações por palavras-chave, o que deixa passar duplicatas escritas com outras palavras, não agrupa temas relacionados e perde contexto em textos longos. Este trabalho constrói um protótipo de triagem semântica com representações vetoriais, busca por similaridade de cosseno e chunking, dividido em quatro entregas:

Entrega	Arquivo	Conteúdo
1 · Representações	análise_comparativa.ipynb	BoW × TF-IDF × embeddings (dois modelos) nos três pares pedidos e em toda a base
2 · Duplicatas	deteccao_duplicatas.ipynb	função detectar_duplicatas, heatmap, escolha do limiar, falsos positivos e negativos
3 · Chunking	chunking_manifestacoes.ipynb	5 textos mais longos, 4 configurações do RecursiveCharacterTextSplitter, PCA e t-SNE
4 · Aplicação	app_ouvidoria.py	Streamlit com as abas Busca, Base Completa, Espaço Vetorial e Chunking
Base de dados	manifestacoes.json	40 manifestações construídas a partir da especificação do enunciado

2. Base de dados

O enunciado descreve o arquivo *manifestacoes.json*, mas ele não foi disponibilizado. Por isso, a base foi construída seguindo a especificação: 40 manifestações (M001 a M040) com os campos *id*, *data*, *categoria_oficial* e *texto*, nas cinco categorias oficiais, com textos de 95 a 646 caracteres. Os pares citados no enunciado (M003, M017, M008, M022 e M031) correspondem às descrições dadas. Foram incluídos **6 pares de duplicatas** (15% da base), cada um marcado por um campo extra *duplicata_de*, que funciona como gabarito para a Entrega 2, e **5 textos longos** (mais de 500 caracteres), um por categoria, cada um com vários subproblemas. Também foram incluídos "vizinhos difíceis": textos do mesmo tema que não são duplicatas, como falta de remédio e falta de médico no mesmo posto, para testar os falsos positivos.

3. Entrega 1 — Representações esparsas × densas

As 40 manifestações foram representadas por BoW e TF-IDF (scikit-learn, com uma lista própria de stopwords em português) e por embeddings de dois modelos multilíngues: *paraphrase-multilingual-MiniLM-L12-v2* (384 dimensões) e *paraphrase-multilingual-mpnet-base-v2* (768 dimensões).

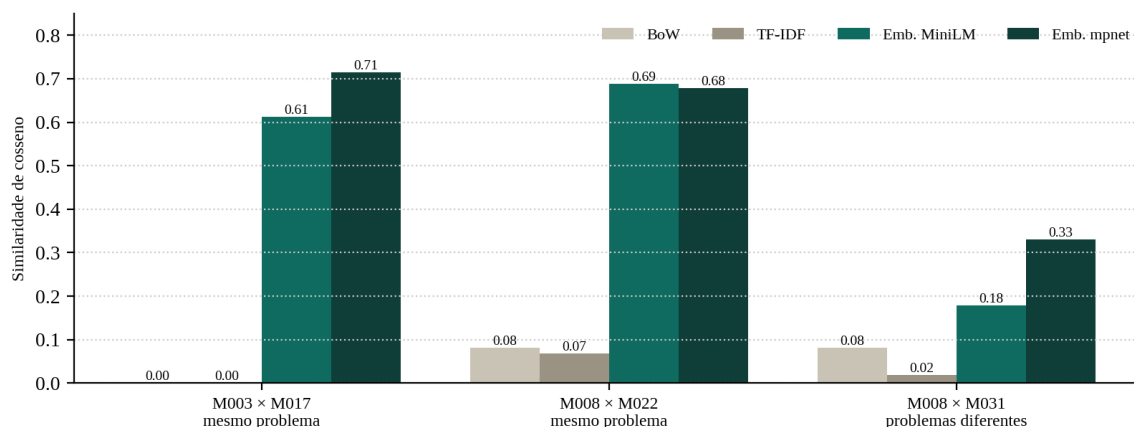


Figura 1 — Similaridade de cosseno nos três pares pedidos, em cada representação.

As representações esparsas falharam exatamente como o enunciado previa. M003 e M017 descrevem o mesmo buraco sem nenhuma palavra em comum, e BoW e TF-IDF dão similaridade **zero**. No BoW, a duplicata M008 × M022 e o par sem relação M008 × M031 empatam em 0,081, porque cada par compartilha uma única palavra irrelevante ("cristo" e "bairro"). Os embeddings deram 0,61 a 0,71 para as duplicatas e 0,18 a 0,33 para o par sem relação. Em toda a base, a diferença entre a média das duplicatas e a média de pares de categorias diferentes foi de 0,21 no BoW, 0,19 no TF-IDF, 0,54 no MiniLM e 0,52 no mpnet. O MiniLM separou um pouco melhor, mesmo sendo o menor, e foi o modelo adotado nas entregas seguintes. Um ponto de atenção: a escala muda de modelo para modelo (o mpnet dá 0,33 para saúde × iluminação, quase o dobro do MiniLM), então limiares não podem ser copiados de um modelo para outro.

4. Entrega 2 — Detecção de duplicatas

A função *detectar_duplicatas(textos, limiar=0.85)* gera os embeddings normalizados, calcula a matriz de similaridade e devolve os pares ($i < j$) acima do limiar, do mais para o menos parecido. Para justificar o limiar, foram analisados a distribuição das 780

similaridades, uma varredura de limiares contra o gabarito e o limiar dinâmico por percentil sugerido no enunciado.

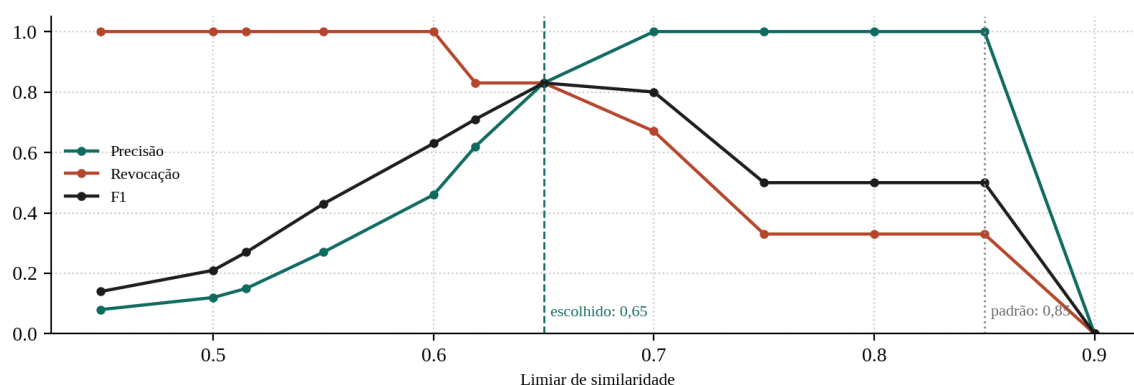


Figura 2 — Precisão, revocação e F1 em função do limiar (MiniLM), comparados com o gabarito de 6 duplicatas.

Com o limiar padrão de 0,85, a função encontrou só 2 das 6 duplicatas: as duplicatas reais deste modelo ficam entre 0,613 e 0,886. O limiar escolhido foi **0,65**, o de maior F1 (0,83), com 5 acertos, 1 falso positivo e 1 falso negativo. O percentil 99 (0,619) chegou perto (F1 de 0,71) e é a alternativa quando não há gabarito. Não existe limiar perfeito: a duplicata menos parecida (0,613) fica abaixo do par não duplicado mais parecido (0,668).

Falso positivo — M020 × M036 (0,668): as duas manifestações longas do bairro Gramame, uma sobre alagamento e outra sobre a escola. Ambas contam que, com a chuva, a estrada vira lama e o ônibus deixa de passar. Resumido num único vetor, cada texto longo vira uma "média" de assuntos, o que motiva a Entrega 3. **Falso negativo — M003 × M017 (0,613):** o mesmo buraco descrito com vocabulário totalmente diferente, sem citar a Av. Brasil. Na prática, o sistema deve **sugerir** duplicatas para um atendente confirmar, e não juntá-las automaticamente.

5. Entrega 3 — Chunking das manifestações longas

As 5 manifestações mais longas foram divididas com o *RecursiveCharacterTextSplitter* (separadores: parágrafo, linha, fim de frase, vírgula e espaço, com *keep_separator="end"*) em quatro configurações de (chunk_size, chunk_overlap):

Configuração	Chunks	Pares com trecho repetido	Coesão consecutiva	Fidelidade ao documento	Vizinho da mesma manifestação
A (200, 0)	23	0 de 18	0,350	0,624	57%
B (200, 50)	23	0 de 18	0,350	0,624	57%
C (350, 70)	13	1 de 8	0,502	0,743	77%
D (350, 180)	17	11 de 12	0,714	0,803	100%

O achado principal foi sobre o overlap. As configurações A e B saíram idênticas: o LangChain primeiro corta o texto em frases e só consegue repetir, no overlap, **frases inteiras** que caibam em chunk_overlap. Como as frases têm de 70 a 175 caracteres, um overlap de 50 não repete nada. A configuração D foi incluída para ter um overlap capaz de repetir frases, e só então o efeito apareceu: a coesão entre chunks consecutivos subiu de 0,502 (C) para 0,714 (D), com o custo de 31% mais chunks. D também preservou melhor o sentido, com a maior fidelidade ao documento e nenhum chunk curto. Em M025, o chunk sobre a falta de cardiologista passa a vir junto com a frase "Ele chegou com dor no peito...", e a denúncia fica compreensível sozinha.

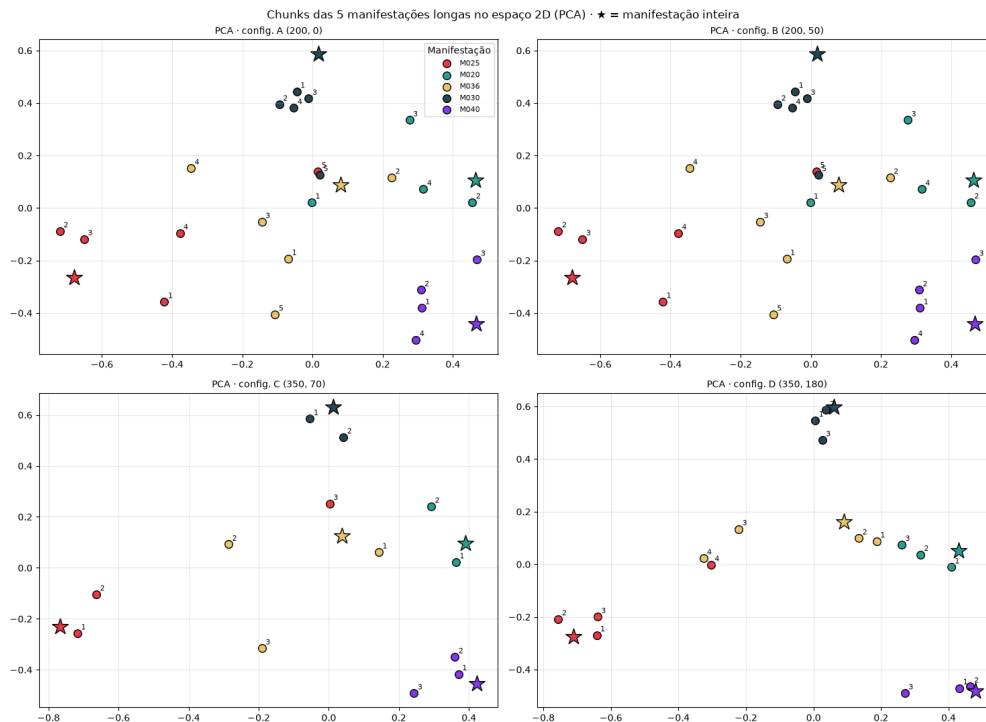


Figura 3 — Chunks das 5 manifestações longas projetados com PCA (★ = manifestação inteira). Em A e B, os pedidos finais de M025 e M030 caem no mesmo ponto; em D, cada manifestação forma um grupo compacto.

No espaço 2D, os chunks de uma mesma manifestação ficaram mais próximos à medida que ganharam contexto. Em A e B, frases curtas de pedido à prefeitura, de denúncias diferentes, ficaram mais próximas entre si do que das próprias denúncias. Em D, o vizinho mais próximo de todo chunk é da mesma manifestação, embora esse valor seja inflado pelas frases repetidas.

6. Entrega 4 — Aplicação Streamlit

O `app_ouvidoria.py` tem as quatro abas pedidas e uma barra lateral com seleção de modelo (os dois multilíngues e o all-MiniLM-L6-v2, em inglês, para comparação), top-K configurável (padrão 5), limiar de duplicata e redução 2D. O modelo usa `st.cache_resource` e os embeddings usam `st.cache_data`. Além do pedido, a aba de busca sugere a categoria por voto ponderado dos resultados e avisa quando a descrição pode ser uma duplicata. O gráfico do espaço vetorial é interativo, e a aba de chunking mostra coesão, trechos repetidos, a matriz de similaridade entre chunks, a projeção 2D e uma busca dentro dos chunks.

Espaço semântico das manifestações — PCA

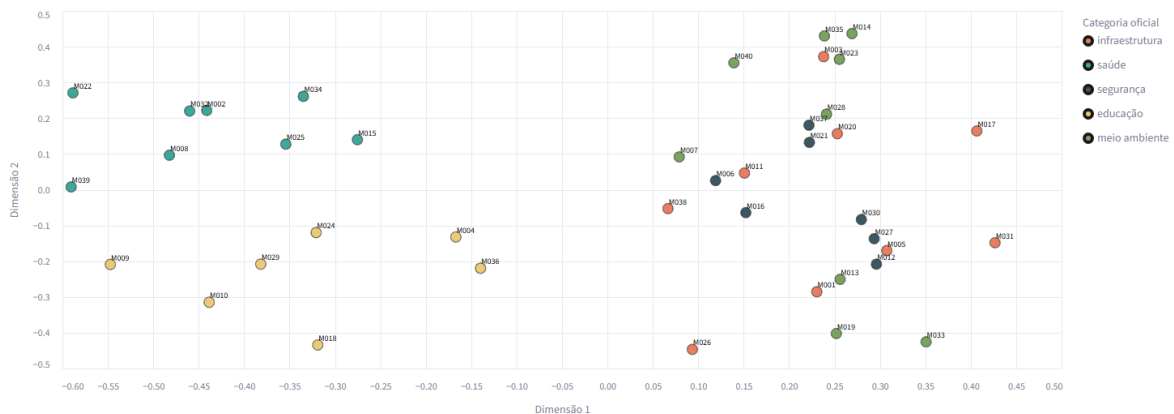


Figura 4 — Aba Espaço Vetorial (MiniLM, PCA), com as manifestações coloridas pela categoria oficial.

Os clusters coincidem com as categorias? Em boa parte, sim: em 88% das manifestações, o vizinho mais próximo é da mesma categoria. Saúde e educação formam grupos bem separados, por terem vocabulário próprio. Já infraestrutura, segurança e meio ambiente se misturam (silhueta de 0,14): o buraco da Av. Brasil (M003) fica ao lado das denúncias de lixo em terrenos, e as câmeras desligadas nos postes (M021, segurança) ficam junto do alagamento de M020. O modelo agrupa pelo assunto concreto, e as categorias oficiais são administrativas.

7. Principais decisões

Decisão	Justificativa
Modelo MiniLM multilíngue nas Entregas 2 a 4	Melhor separação na Entrega 1, 3× menor e mais rápido que o mpnet
Limiar de duplicata 0,65 (e não 0,85)	Maior F1 no gabarito; 0,85 perdia 4 das 6 duplicatas neste modelo
Configuração D (350, 180) no chunking	Único overlap que repete frases; maior coesão e fidelidade

Stopwords em português escritas à mão	O scikit-learn só oferece a lista em inglês
Sugerir, e não fundir, duplicatas e categorias	Os erros são inevitáveis nas fronteiras entre temas

8. Dificuldades

Base de dados ausente: o JSON teve de ser construído, o que exigiu cuidado para manter os pares do enunciado e incluir casos difíceis. **Modelo inexistente:** o *BAAI/bge-small-pt-v1.5*, sugerido no enunciado, não foi encontrado no Hugging Face (só há versões em inglês e chinês), e foi substituído pelo mpnet multilíngue. **Stopwords:** o parâmetro `stop_words="portuguese"`, usado no material de apoio, gera erro no scikit-learn. **Overlap sem efeito:** a primeira bateria da Entrega 3 deu resultados idênticos para overlap 0 e 50, e foi preciso entender o funcionamento interno do splitter para desenhar uma configuração que testasse o overlap de verdade. **Ambiente:** no Windows, o download dos modelos gerava avisos de cache e barras de progresso que poluíam os notebooks; eles foram desativados por variáveis de ambiente.

9. Aprendizados

A principal lição é que **representações densas resolvem o problema de vocabulário** que derruba BoW e TF-IDF, mas trazem outro: a escala de similaridade depende do modelo, e todo limiar precisa ser calibrado com dados rotulados, nunca copiado. A segunda é que **parâmetros de chunking só fazem sentido conhecendo a ferramenta:** o mesmo overlap de 50 caracteres pode não ter efeito nenhum, dependendo de como o texto é quebrado. Por fim, os erros encontrados (textos longos que se parecem por um detalhe, categorias administrativas que não seguem o assunto do texto) mostram que a triagem semântica funciona melhor como apoio ao atendente do que como decisão automática.